

Research-backed architecture for an agentic coding environment for your low-code workflow platform

You are building exactly the sort of product where coding agents can be useful but also dangerous: a multi-repository platform with a drag-and-draw UI, workflow orchestration, mixed JavaScript and Java code, delivery artefacts spread across Azure DevOps, Confluence, Figma, HTML prototypes, and several repos with different conventions. The strongest conclusion from both recent software-engineering research and current industrial practice is that the winning architecture is **not** a single giant prompt. It is a layered system made of thin always-on repository instructions, scoped project instructions, on-demand skills, deterministic hooks, and tightly-scoped access to external systems through standard interfaces such as MCP. Research on software-engineering agents also points in the same direction: good performance depends on repository-level context, explicit memory management, and mechanisms that keep the right context in play while keeping irrelevant context out. ¹

A second conclusion is just as important for your case: the “knowledge” that agents need is broader than code. OpenAI’s own agent-first engineering write-up emphasises that agents become far more effective when the UI, logs, metrics, execution plans, and repository knowledge are all made legible to the agent, rather than remaining buried in separate tools or tacit team knowledge. In other words, for a workflow-builder product, the problem is not merely “how do I prompt the model?” but “how do I make the product, delivery process, and design intent inspectable, navigable, and enforceable?” ²

Because I cannot inspect your actual Azure DevOps, Confluence, Figma, or repositories from here, the report below is a research-backed reference architecture rather than a diagnosis of your current implementation.

What the evidence says

The academic literature has moved well beyond simple “code generation” and now treats software-engineering agents as systems with **perception, memory, and action**. A recent survey of agents in software engineering frames the field this way explicitly, while a later survey of agentic issue resolution organises the space around benchmarks, techniques, and empirical studies. That framing matters for your platform because it implies that instructions alone will never be enough: the agent needs a disciplined way to perceive the relevant artefacts, maintain durable working memory, and act through tools with feedback. ³

Repository-level context is now a first-class design requirement. The 2023 Repo-Level Prompt Generation paper showed that bringing in relevant context from the wider repository, rather than only the current file, materially improved code generation; it specifically argues that relevant code context may come from imports, parent classes, neighbouring files, and internal API documentation. More recent work pushes this further. RepoGraph, published at ICLR 2025, argues that repository-level understanding is essential for real software tasks and introduces a line-, file-, and repo-level graph that serves as repository-wide navigation for the agent. For your multi-repo environment, this strongly

supports building a navigable map of repos, modules, dependencies, design system artefacts, and boundaries, rather than relying on natural-language instructions alone. 4

Recent agent papers also show that “more context” is not the same thing as “better context”. SemAgent finds that many existing agents over-focus on locally suspicious lines and miss issue semantics, code semantics, and execution semantics, producing over-fitted patches. CAT, a 2025 paper on long-horizon SWE agents, proposes structured context with stable task semantics, condensed long-term memory, and high-fidelity short-term interactions; it reports materially better results on SWE-Bench-Verified than static compression baselines. A June 2026 study on long-horizon tool-using agents similarly reports that selective retention plus summarisation can outperform full-history retention while using fewer tokens and less time. The practical implication is clear: your environment should favour **targeted retrieval, milestone summaries, and scoped memory**, not append-only chat history or ever-growing instruction files. 5

Benchmarks also suggest you should not assume even performance across your stack. SWE-PolyBench contains 2,110 repository-level tasks across Java, JavaScript, TypeScript, and Python, and the authors report uneven agent performance across languages, task types, and complexity. That is directly relevant to your Java plus JavaScript estate: you will need your own internal evaluation suite spanning UI, backend, orchestration, and cross-repo changes, because a setup that looks good on JavaScript-only UI work may perform much worse on Java services or orchestration logic. 6

A practical architecture for your environment

The most robust architecture for your situation has five layers: **always-on instructions, role-specific agents, on-demand skills, deterministic hooks**, and **external context connectors**. This aligns with current product behaviour across Codex, Claude, and VS Code customisation features. `AGENTS.md` is for persistent repository instructions. Custom agents or subagents are for role and tool selection. Skills are for reusable specialised workflows. Hooks are for deterministic enforcement. MCP is for clean access to external systems and live data. 7

A good way to think about the layers is this:

Layer	Purpose	Typical artefacts
Always-on instructions	Stable norms and repository map	<code>AGENTS.md</code> , nested <code>AGENTS.md</code> , <code>ARCHITECTURE.md</code>
Role-specific agents	Limit tools and behaviour by task	planner, UI implementer, backend implementer, reviewer
Skills	Repeatable specialised workflows	<code>SKILL.md</code> , scripts, references, templates
Hooks	Hard enforcement and automation	pre/post tool hooks, approval hooks, stop hooks
Context connectors	Live access to work systems	Azure DevOps MCP, Atlassian Rovo MCP, Figma MCP, internal MCP servers

For your estate, the external connectors should start with the places where work actually happens. Microsoft’s Azure DevOps MCP server gives agents secure access to work items, pull requests, builds, test plans, and documentation; its own guidance explicitly recommends starting with the remote MCP

server and even suggests putting a project instruction in your repo telling the assistant to check Azure DevOps tools when relevant. Atlassian's Rovo MCP server gives secure real-time access to Jira, Confluence, Bitbucket, and related artefacts, while Atlassian's own guidance for subagents shows that narrowing custom knowledge sources improves relevance and accuracy. Figma's MCP server gives agents direct design context, lets them extract variables, components, and layout data, and can generate code context for a selected frame or layer; Figma also points to Code Connect as the way to keep generated output aligned with real components. ⁸

The key architectural move is to use MCP the way the protocol is designed to be used. The MCP specification separates **resources**, **prompts**, and **tools**. In your case, that suggests a clean mapping: Confluence pages, design-system references, repo indices, and generated summaries should be exposed as **resources**; repeatable workflows such as "turn a work item plus design into an execution plan" should be exposed as **prompts**; and actions such as "fetch current sprint work", "read Figma frame context", "create a PR status", or "open a wiki page update" should be exposed as **tools**. This is cleaner and more maintainable than shoving everything into one agent instruction file. ⁹

Do not wire every possible system in on day one. Official Codex guidance advises adding tools only when they unlock a real workflow and explicitly recommends starting with one or two tools that remove a manual loop you already do often. For your platform, the first two high-value loops are likely to be: **Azure DevOps sprint/work-item context** and **Figma design context for UI changes**. Confluence should follow quickly as the design and architectural narrative layer. ¹⁰

How to structure AGENTS, skills, and roles

The cleanest mental model is:

- **AGENTS.md** tells the agent how to behave in this part of the codebase all the time.
- Custom agents or subagents decide which persona, tools, and hand-offs to use for a task.
- Skills package repeatable workflows and specialist knowledge for on-demand use.
- Hooks enforce behaviour mechanically when instructions are not enough.

That distinction is crucial because many teams overload **AGENTS.md** and then wonder why the agent becomes erratic. OpenAI's own guidance says **AGENTS.md** files are loaded before work begins, layered from global to repository to deeper directories; later, more specific directories override earlier ones. VS Code likewise supports **AGENTS.md** for shared instructions and nested **AGENTS.md** files for subfolder-specific rules in monorepos. OpenAI's own harness-engineering write-up goes further and says that a single giant **AGENTS.md** failed for predictable reasons: it consumed scarce context, buried what mattered, went stale quickly, and was hard to verify mechanically. Their solution was to keep **AGENTS.md** short and use it as a **map**, not an encyclopedia. ¹¹

For your platform, I would structure the repository or repo family like this:

```
AGENTS.md
ARCHITECTURE.md
docs/
  index.md
  design-docs/
  exec-plans/
    active/
    completed/
```

```

product-specs/
references/
generated/
repos/
  ui/
    AGENTS.md
    skills/
      implementing-ui-from-figma/
      reviewing-accessibility/
  backend/
    AGENTS.md
    skills/
      java-service-change/
      api-contract-check/
  orchestration/
    AGENTS.md
    skills/
      workflow-runtime-change/
  shared/
    AGENTS.md
.vscode/
  mcp.json
.github/
  hooks/
  agents/
    planner.agent.md
    ui-implementer.agent.md
    backend-implementer.agent.md
    reviewer.agent.md

```

Your **solution-level** `AGENTS.md` should stay short, probably around one hundred lines rather than hundreds. It should describe the repo or repo-family map, top-level commands, architectural boundaries, where source-of-truth docs live, how to choose the right subproject, how to link work to Azure DevOps items, and which deeper docs to consult next. It should not contain detailed UI conventions, Java package rules, database migration sequences, or design-system minutiae; those belong deeper in project-level instructions or in skill references. This is precisely the pattern OpenAI describes in its own agent-first engineering work. ¹²

Your **project-level** `AGENTS.md` files should sit as close as possible to the code they govern. The UI instruction file should name framework conventions, state-management patterns, design-system usage rules, accessibility expectations, test and storybook commands, visual-regression checks, and the fact that Figma context must be consulted before changing certain component classes. The backend instruction file should focus on Java package boundaries, validation, error handling, schema evolution, contract tests, and performance or telemetry conventions. The orchestration instruction file should focus on workflow definitions, compatibility rules, idempotency, saga or retry semantics, and how runtime behaviours are verified. OpenAI explicitly recommends putting guidance in the closest directory where it applies and using `AGENTS.md` as a feedback loop by updating it when recurring corrections occur. ¹³

Your **custom agents** should represent roles rather than domains. VS Code's custom-agent model is especially useful here because it supports different tools, instructions, and hand-offs per agent. A practical minimum set would be: a **planner** with read-only tools; a **UI implementer** with Figma and UI-repo edit access; a **backend implementer** with Java-repo edit access; and a **reviewer** with diff, test, and policy-check tools. VS Code also supports hand-offs, so a planner can hand off to implementation, and implementation can hand off to review with the relevant context already carried forward. Least-privilege is built into this model: planning agents can be read-only, while implementation agents get editing tools only where needed. ¹⁴

Your **skills** should cover repeatable workflows, not permanent rules. Official guidance from both OpenAI and Anthropic is very aligned here. Skills should be concise, well-scoped, and automatically discoverable through a good description that says what the skill does and when to use it. Anthropic recommends concise bodies, progressive disclosure into separate reference files, explicit workflows for complex tasks, feedback loops for quality-critical tasks, and testing with all models you plan to use. OpenAI similarly advises keeping each skill scoped to one job, starting from two or three concrete use cases, and turning a workflow into a skill when you keep reusing the same prompt or correcting the same behaviour. ¹⁵

For your platform, the first useful skills would probably be:

- implementing-ui-from-figma
- updating-design-system-consumers
- java-service-change
- api-contract-check
- workflow-runtime-change
- execution-plan-from-work-item
- pr-review-against-architecture
- traceability-and-release-notes

Each should do one thing well, include the right trigger phrases, and carry scripts or references only when they improve reliability. ¹⁶

How to distil artefacts into a maintainable knowledge base

The biggest design decision is this: **your business source of truth can remain in Azure DevOps, Confluence, and Figma, but your agent-operational source of truth should become a versioned, repo-local knowledge base.** OpenAI's harness-engineering article is very clear on the principle: what the agent cannot access in context effectively does not exist. Their answer was to treat the repository's structured docs directory as the system of record for the agent, with `AGENTS.md` acting as a short map into deeper material such as design docs, product specs, execution plans, generated references, and quality or reliability documents. They also mechanically validated that knowledge base and ran a recurring "doc-gardening" agent to fix stale or obsolete documentation. ¹⁷

For your platform, I would separate content into three classes:

Class	Meaning	Examples
Authoritative external artefacts	The official business/project source	Azure DevOps work items, Confluence pages, Figma files

Class	Meaning	Examples
Repo-local canonical operational docs	What agents rely on during coding	<code>docs/index.md</code> , architecture maps, execution plans, ADRs, domain references
Generated derivative artefacts	Rebuilt from source systems	summaries, schema refs, dependency maps, component inventories, “what changed” digests

That gives you a maintainable distillation pipeline:

Ingest deltas. Use MCP or API connectors to detect changes in current sprint items, new or edited Confluence pages, changed Figma frames, and repository commits or PRs. Azure DevOps MCP and Atlassian RoVo MCP are designed for exactly this kind of live access, and Codex’s own guidance on “setting up a teammate” recommends connecting the tools where work happens so the agent can notice what changed. ¹⁸

Normalise and classify. Every changed artefact should be classified by domain, for example `ui`, `backend`, `workflow-runtime`, `shared-contracts`, or `delivery-process`. The output of this step should be lightweight metadata rather than prose: IDs, owners, affected repos, affected packages, freshness timestamps, and links back to source artefacts. This keeps later retrieval targeted rather than noisy. The MCP pattern of resources, prompts, and tools fits this well because resources can carry structured contextual data without forcing the model to regenerate it every time. ¹⁹

Distil into stable repo-local documents. Not every source page should be pasted into the repo. Instead, create durable operational docs such as `docs/product-specs/workflow-canvas.md`, `docs/references/ui-design-system.md`, `docs/exec-plans/active/<ticket>.md`, or `docs/generated/component-inventory.md`. OpenAI’s documented pattern is especially helpful here: active plans, completed plans, technical debt, generated references, and design documentation should all be versioned and cross-linked so agents can operate without depending on hidden context outside the repo. ¹⁷

Preserve provenance and freshness. Every derived page should say where it came from, when it was refreshed, and which systems it reflects. Anthropic’s skill guidance explicitly warns against embedding time-sensitive rules without handling staleness, and recommends moving older patterns into clearly labelled historical sections instead of leaving ambiguous dated guidance in the main flow. The same principle applies to your generated documentation: freshness should be visible and auditable. ²⁰

Maintain incrementally rather than rewriting everything. Recent work on long-horizon agents and context engineering strongly supports incremental summarisation over full-history retention. CAT proposes stable task semantics plus condensed long-term memory, Anthropic’s context-engineering guidance compares memory, compaction, and tool clearing for long-running agents, and the June 2026 efficiency study shows that pruning plus summarisation can beat full-history retention. Concretely, that means you should build milestone summaries such as “why this UI pattern exists”, “what changed in this sprint”, or “what constraints apply to workflow runtime vNext”, and refresh only the affected summaries when upstream artefacts change. ²¹

A practical distillation cadence would look like this:

- on every merged PR, refresh affected repo maps, dependency indices, and execution-plan status;
- nightly, refresh sprint summaries, “changed Confluence pages”, and “changed Figma frames” digests;

- weekly, run a doc-gardening agent to identify stale references and open repair PRs;
- before a release, regenerate release notes, contract inventories, and traceability reports.

That pattern aligns well with OpenAI's "durable teammate thread" idea and with its own use of long-running threads and automations to notice changed docs, blocked hand-offs, and decisions requiring judgement. ²²

How to make the agent use the right knowledge in the right place

This is the heart of your question: how do you ensure UI knowledge is used when changing UI, backend knowledge when changing backend, and so on?

The answer is to combine **scope**, **discovery**, and **enforcement**.

Scope comes from path-specific instructions. Codex injects instruction files from the global level through the repo root down to the current working directory, with later and deeper directories overriding earlier guidance. VS Code likewise supports nested `AGENTS.md` files for different folders in a monorepo and can collect customisations up the folder hierarchy to the repository root. That means the first and most reliable control is simply to place the right instructions in the right directories and keep them local. If a change is in `ui/`, the closest `AGENTS.md` should already carry the relevant design-system and testing rules. ²³

Discovery comes from skill descriptions and role selection. Anthropic's skill guidance says the `description` field is what enables skill discovery and should include both what the skill does and when to use it. OpenAI makes the same point, advising descriptions that match the phrases users actually use. So your UI skill descriptions should literally mention terms such as "React component", "workflow canvas", "drag-and-drop", "design-system token", "Figma frame", and "accessibility". Your backend skills should mention "Java service", "API contract", "schema change", "idempotency", "orchestration runtime", and similar trigger terms. On top of that, role-specific custom agents make discovery more robust by selecting the right tools and instructions for the task from the start. ²⁴

Enforcement comes from hooks and policy. This is where many teams improve reliability dramatically. VS Code explicitly distinguishes hooks from instructions: hooks are deterministic, code-driven automation that execute at lifecycle points with guaranteed outcomes, whereas instructions merely guide behaviour. Codex hooks provide very similar control. `PreToolUse` can intercept and deny or rewrite supported tool calls; `PermissionRequest` can automatically allow or deny requests that would otherwise surface for approval; `PostToolUse` can run follow-up automation; `SessionStart` and `SubagentStart` can inject additional developer context; and `Stop` hooks can force another pass before the run ends. ²⁵

For your platform, I would use hooks in very concrete ways:

- A **session-start hook** loads the active sprint work item, linked execution plan, and relevant architectural references before coding starts. Codex and VS Code both support session-start style context injection. ²⁶
- A **pre-tool hook** blocks dangerous or out-of-scope operations, for example editing backend code from the UI agent, touching generated files directly, or bypassing the plan for a multi-repo change. Both VS Code and Codex document this pattern. ²⁷

- A **post-tool hook** runs local formatters, linters, targeted unit tests, accessibility checks, or component snapshots only for the files that changed. VS Code documents `PostToolUse` formatting hooks directly, and Codex supports post-tool continuation logic. ²⁸
- A **stop hook** checks whether the output is reviewable and whether required follow-up work is missing. Codex's `Stop` hooks can continue the run with a new prompt such as “run one more pass over failing tests” or “link the change back to the work item and update docs before stopping”. ²⁹

The UI path deserves special treatment in your product because “change the UI” really means “change code in line with design intent”. Figma’s MCP server exists precisely to solve that gap. It can extract variables, components, and layout data into the IDE; it can give design context for a frame or layer; and Figma explicitly points to Code Connect as the way to reuse the actual components from your codebase. Combined with a UI-specific skill and a post-change validation hook, this lets the agent consult design context before editing and then validate the result afterwards. That is far stronger than hoping the model “remembers” your design rules from a prompt. ³⁰

You should also enforce the “right place” through your delivery platform, not just inside the agent. Azure DevOps branch policies support path filters, required reviewers, automatically included code reviewers, build validation, and external service PR statuses. That means you can require UI reviewers or UI-specific validation only when files under the UI paths change, require backend approval for backend paths, and require external policy results from your own architectural checker or agent reviewer. Azure DevOps also supports linking work items to commits, builds, pull requests, tests, and wiki pages for traceability, which gives you a strong audit trail from sprint plan to code to test to documentation. ³¹

One further industrial lesson is worth carrying over: agents are much stronger when the application itself is legible. OpenAI reports wiring Chrome DevTools Protocol into the agent runtime so the agent could inspect DOM snapshots, screenshots, and navigation, and exposing logs, metrics, and traces through an observability stack the agent could query directly. For a workflow-builder platform, that suggests adding browser automation plus local observability to your agent environment, especially for UI-heavy or orchestration-heavy changes. It is not enough for the agent to edit files; it should be able to run the canvas, inspect the flow editor, check browser errors, and reason over runtime traces. ²

Governance, evaluation, and rollout

You should start from a conservative trust posture. OpenAI’s security guidance for Codex says the local agent defaults to no network access, uses an OS-enforced sandbox, and should keep approvals and sandboxing tight by default until the need to loosen them is clear. VS Code’s custom-agent model reinforces the same principle by letting you create read-only planning agents and more capable implementation agents only where necessary. Anthropic likewise recommends extensive testing in sandboxed environments with appropriate guardrails for autonomous agents. For your environment, that means planning, research, and documentation agents should normally be broader and safer, while implementation agents should be narrower and more heavily checked. ³²

Evaluation should be built in from the beginning. Anthropic’s skill guidance recommends creating multiple evaluations, testing with the models you actually plan to use, and iterating based on observed behaviour in real workflows. SWE-PolyBench’s cross-language benchmark is a reminder that mixed-language estates behave unevenly. In practice, you should create a private task suite covering at least: UI component change from Figma, workflow-canvas behavioural change, Java service contract update, orchestration logic fix, shared-library refactor, and a cross-repo feature slice. Measure not only task

completion, but also wrong-file edits, review burden, test breakage, traceability completeness, and how often hooks or policies had to intervene. ³³

A sensible rollout would happen in three waves.

First, establish the control plane: root and nested `AGENTS.md`, three or four role-specific agents, Azure DevOps and Figma MCP access, and a minimal set of hooks for formatting, linting, and test execution. This gives you immediate gains without overbuilding. ³⁴

Second, build the knowledge base and distillation loop: `docs/index.md`, execution plans, product specs, generated references, freshness metadata, and a doc-gardening workflow. This is the stage where you turn scattered artefacts into durable operational context the agent can actually use. ³⁵

Third, increase application legibility and review automation: browser automation, UI screenshots and DOM inspection, observability access, architecture status checks, and agent-assisted review loops. OpenAI's own engineering notes suggest that once the environment is legible and enforceable, agent-to-agent review and much higher throughput become feasible. ³⁶

The strongest single recommendation I can give, after looking across the research and the industrial material, is this: **treat agent effectiveness as an environment-design problem, not a prompt-writing problem**. Keep solution-level guidance short and map-like. Push domain truth into repo-local, versioned docs. Use nested `AGENTS.md` for locality, skills for repeatable jobs, custom agents for role and tool selection, hooks for anything that must be enforced, MCP for live external context, and branch/policy controls so the delivery platform backs the same boundaries. That architecture is the most credible way to make an agent change the right code, with the right knowledge, in the right place, while staying maintainable as your low-code platform grows. ³⁷

¹ ³ [2409.09030] Agents in Software Engineering: Survey, Landscape, and Vision

<https://arxiv.org/abs/2409.09030>

² ¹² ¹⁷ ³⁵ ³⁶ ³⁷ Harness engineering: leveraging Codex in an agent-first world | OpenAI

<https://openai.com/index/harness-engineering/>

⁴ Repository-Level Prompt Generation for Large Language Models of Code

<https://proceedings.mlr.press/v202/shrivastava23a/shrivastava23a.pdf>

⁵ [2506.16650] SemAgent: A Semantics Aware Program Repair Agent

<https://arxiv.org/abs/2506.16650>

⁶ [2504.08703] SWE-PolyBench: A multi-language benchmark for repository level evaluation of coding agents

<https://arxiv.org/abs/2504.08703>

⁷ ¹¹ ³⁴ Custom instructions with AGENTS.md – Codex | OpenAI Developers

<https://developers.openai.com/codex/guides/agents-md>

⁸ ¹⁸ Enable AI assistance with the Azure DevOps MCP Server - Azure Boards | Microsoft Learn

<https://learn.microsoft.com/en-us/azure/devops/mcp-server/mcp-server-overview?view=azure-devops>

⁹ ¹⁹ Specification - Model Context Protocol

<https://modelcontextprotocol.io/specification/2025-06-18>

¹⁰ ¹⁶ ³² Best practices – Codex | OpenAI Developers

<https://developers.openai.com/codex/learn/best-practices>

- 13 Customization – Codex | OpenAI Developers
<https://developers.openai.com/codex/concepts/customization>
- 14 Custom agents in VS Code
<https://code.visualstudio.com/docs/agent-customization/custom-agents>
- 15 Agent Skills - Claude Platform Docs
<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>
- 20 24 33 Skill authoring best practices - Claude Platform Docs
<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices>
- 21 [2512.22087] Context as a Tool: Context Management for Long-Horizon SWE-Agents
<https://arxiv.org/abs/2512.22087>
- 22 Set up a teammate | Codex use cases
<https://developers.openai.com/codex/use-cases/proactive-teammate>
- 23 Prompt guidance | OpenAI API
<https://developers.openai.com/api/docs/guides/prompt-guidance>
- 25 27 28 Agent hooks in Visual Studio Code (Preview)
<https://code.visualstudio.com/docs/agent-customization/hooks>
- 26 29 Hooks – Codex | OpenAI Developers
<https://developers.openai.com/codex/hooks>
- 30 Introduction | Developer Docs
<https://developers.figma.com/docs/figma-mcp-server/>
- 31 Git branch policies and settings - Azure Repos | Microsoft Learn
<https://learn.microsoft.com/en-us/azure/devops/repos/git/branch-policies?view=azure-devops>